

Indian Institute of Information Technology Bhopal



Digital Image Processing

Level 3: Document Scanner



विद्या ददाति विनयं

Submitted by:	Guided by:
Aryan Chouhan and Yuvraj Singh	Dr. Supriya Agrawal
23U03121 and 23U03086	Prof IIIT Bhopal

L3 Project: Document Scanner using OpenCV

AIM

To design and implement a Document Scanner using Python and OpenCV that automatically detects, crops, and enhances documents from captured or uploaded images, simulating the functionality of mobile scanning apps.

THEORY

1. Overview

Document scanning involves detecting the boundaries of a document within an image, applying perspective transformation, and enhancing the output to achieve a clear, flat, and readable scanned copy. The process uses fundamental computer vision techniques like edge detection, contour approximation, and perspective correction.

2. Steps Involved in Document Scanning

- a) **Preprocessing:** Convert the image to grayscale and apply Gaussian blur to remove noise.
- b) **Edge Detection:** Use Canny Edge Detection to identify document boundaries.
- c) **Contour Detection:** Find the largest contour approximated to four corners representing the document edges.
- d) **Perspective Transformation:** Apply a perspective warp to obtain a top-down view of the document.
- e) **Contrast Enhancement:** Improve brightness and contrast for better readability.

3. Mathematical Concept: Perspective Transform

Perspective transformation maps the four corner points of a document to a rectangle in a new coordinate system.

If the document corners are A, B, C, D in the input image and A', B', C', D' are in the output image, the transformation matrix 'M' satisfies:

$$[x', y', 1]^T = M * [x, y, 1]^T$$

CODE

```
import cv2 as cv
import numpy as np

def four_corners_document(img: np.ndarray, processing_height_in_px: int = 800):
    # Resize image for processing
    ratio = img.shape[0] / processing_height_in_px
    resized = cv.resize(img, (int(img.shape[1] / ratio), processing_height_in_px))
    gray = cv.cvtColor(resized, cv.COLOR_BGR2GRAY)
    blurred = cv.GaussianBlur(gray, (5, 5), 0)
    edges = cv.Canny(blurred, 75, 200)

    # Find contours and get the largest one
    contours, _ = cv.findContours(edges.copy(), cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)
```

```

contours = sorted(contours, key=cv.contourArea, reverse=True)[:5]

for c in contours:
    peri = cv.arcLength(c, True)
    approx = cv.approxPolyDP(c, 0.02 * peri, True)
    if len(approx) == 4:
        doc_cnt = approx
        break

return doc_cnt.reshape(4, 2) * ratio

def order_points(pts):
    rect = np.zeros((4, 2), dtype="float32")
    s = pts.sum(axis=1)
    rect[0] = pts[np.argmin(s)]
    rect[2] = pts[np.argmax(s)]
    diff = np.diff(pts, axis=1)
    rect[1] = pts[np.argmin(diff)]
    rect[3] = pts[np.argmax(diff)]
    return rect

def scan_document(image_path):
    img = cv.imread(image_path)
    pts = four_corners_document(img)
    rect = order_points(pts)

    (tl, tr, br, bl) = rect
    widthA = np.linalg.norm(br - bl)
    widthB = np.linalg.norm(tr - tl)
    heightA = np.linalg.norm(tr - br)
    heightB = np.linalg.norm(tl - bl)

    maxWidth = int(max(widthA, widthB))
    maxHeight = int(max(heightA, heightB))

    dst = np.array([[0, 0], [maxWidth - 1, 0],
                    [maxWidth - 1, maxHeight - 1], [0, maxHeight - 1]], dtype="float32")

    M = cv.getPerspectiveTransform(rect, dst)
    warped = cv.warpPerspective(img, M, (maxWidth, maxHeight))

    # Convert to grayscale and enhance
    gray_warped = cv.cvtColor(warped, cv.COLOR_BGR2GRAY)
    scanned = cv.adaptiveThreshold(gray_warped, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv.THRESH_BINARY, 11, 2)

    return scanned

```

Input Images:

1. Input1

Four discrete intensity

$$S_k = T(u) = (L-1) \sum_{k=0}^{K-1} R_k(u)$$

Spatial Filtering: Spatial filters are placed based on a function of its neighbors

Low pass filter $\rightarrow$ passes low freq. $\rightarrow$ smooths/blurs the image

High pass filter $\rightarrow$ passes high freq. $\rightarrow$ sharpens edges

Linear spatial filtering

$$g(x,y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s,t) f(x-s, y-t)$$

3x3 kernel

$$W = \begin{bmatrix} w(-1,-1) & w(-1,0) & w(-1,1) \\ w(0,-1) & w(0,0) & w(0,1) \\ w(1,-1) & w(1,0) & w(1,1) \end{bmatrix}$$

Spatial convolution vs correlation

0 0 0 0 0
0 0 0 0 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 0

padding

0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0

0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

Result convolution using padded w

0 8 7
6 5 4
3 2 1

Result convolution using w

0 0 0 0 0
0 8 8 7 0
0 6 5 4 0
0 3 2 1 0
0 0 0 0 0

Result convolution using padded w

0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

Separable discrete kernels

- A 2D function $G(x,y)$ is said to be separable if $G(x,y) = G_1(x) \cdot G_2(y)$
- A separable kernel of size $M \times N$ can be expressed as $W = V \times W^T$
- e.g. $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$

Smoothing spatial filters

Linear Filters: Mean/Box, weighted average, Gaussian Filter

Non-Linear Filters: median filter, Max filter, Min filter

Median filter: preserve the sharp edges

Max filter: Finding min of all pixel in kernel

Min filter: Finding max of all pixel in kernel

Median filter: Better than salt and pepper noise

Blurting $\rightarrow$ pixel averaging (integration)

Sharpening $\rightarrow$ spatial differentiation

First order derivatives of 1D fn

$$\frac{df}{dx} = f(x+1) - f(x)$$

2nd order

$$\frac{d^2f}{dx^2} = f(x+2) + f(x) - 2f(x+1) \text{ or } f(x+1) + f(x-1) - 2f(x)$$

Laplacian filter $\frac{d^2f}{dx^2} + \frac{d^2f}{dy^2}$

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \text{ or } \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

2. Input2



Output Images:

1. Output1

Four discrete intensity

$$S_k = T(x) = (L-1) \sum_{k=0}^{L-1} P_k(x)$$

Spatial Filtering: Sp modifies each pixel based on a function of its neighbors

Low pass filter $\rightarrow$ passes low freq. $\rightarrow$ smooths/blurs the image

High pass filter $\rightarrow$ passes high freq. $\rightarrow$ sharpens edges

Linear spatial filtering

$$g(x,y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s,t) f(x+s, y+t)$$

3x3 kernel

$$w = \begin{bmatrix} w(-1,-1) & w(-1,0) & w(-1,1) \\ w(0,-1) & w(0,0) & w(0,1) \\ w(1,-1) & w(1,0) & w(1,1) \end{bmatrix}$$

Spatial convolution vs correlation

0 0 0 0 0
0 0 0 0 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 0

padding

0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0

W = $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ Rotated W $\begin{bmatrix} 3 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$

result
convolution using w

0 0 0 0 0
0 8 8 7 0
0 6 5 4 0
0 3 2 1 0
0 0 0 0 0

result
convolution using
rotated w

0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

Separable discrete kernels

- A 2D function $g(x,y)$ is said to be separable if $g(x,y) = g_1(x) \cdot g_2(y)$
- A separable kernel of size $m \times n$ can be expressed $W = V \times W^T$

eg. $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$

Smoothing spatial filters

Linear Filters

- Mean/Box
- weighted average
- Gaussian Filter

Non-Linear Filters

- Median filter
- Max filter
- Min filter

preserve the sharp edges

Bad for salt and pepper noise

Blurring $\rightarrow$ pixel averaging (integration)

Sharpening $\rightarrow$ spatial differentiation

First order derivatives of 2D fn

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

2nd order

$$\frac{\partial^2 f}{\partial x^2} = f(x+2) + f(x) - 2f(x+1) \text{ or } f(x+1) + f(x-1) - 2f(x)$$

Laplacian filter $\nabla^2 f$

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \text{ or } \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

2. Output2



RESULT

1. The algorithm successfully detected the document boundary from the input image.
2. The perspective transformation produced a top-down, properly aligned version of the document.
3. Adaptive thresholding enhanced the clarity and readability, resembling a scanned copy.
4. Output image saved as 'Scanned_Output.jpg'.

LEARNINGS

- Learned how to detect document contours using edge and contour detection.
- Understood the process of applying perspective transformation for top-down view extraction.
- Learned how adaptive thresholding improves contrast and text visibility.
- Gained practical experience in using OpenCV functions like `findContours()`, `getPerspectiveTransform()`, and `warpPerspective()`.
- Built a mini document scanning system similar to mobile scanning applications.